

RegEx, jQuery & PHP Validation – Full Notes

RegEx, jQuery Validation & PHP (Functions, Arrays, Superglobals)

1. How to think about RegEx

A RegEx is just a pattern that says:

“Among all possible strings, I only want the strings that look like this.”

If a string fits the pattern → match = true

If not → match = false

Very similar to how you might describe people:

“Must be 18-25 years old, male or female, and live in X city”

That’s a pattern for humans.

RegEx is a pattern for text.

2. The basic building blocks

2.1. Literal characters

Simplest regex: just letters.

Pattern:

hello

Matches:

- hello

Does not match:

- Hello (capital H)
- hello123
- xhello

Because we haven’t told it anything about upper case, numbers, or extra characters.

2.2. Character classes: []

[abc] = one character that is either a or b or c.

Examples:

- Pattern: [abc]
- Matches: "a", "b", "c"
- Not match: "d"

Range examples:

- [0-9] = any digit 0 to 9
- [A-Z] = capital letters
- [a-z] = small letters
- [A-Za-z] = any letter

2.3. Shorthand classes: \d, \w, \s

In many regex flavors (including JS/PHP):

- \d = digit → [0-9]
- \w = "word character" → [A-Za-z0-9_]
- \s = whitespace → space, tab, newline, etc.

Example:

- Pattern: \d\d\d
- Matches: "123", "999"
- Not: "12", "abcd"

2.4. Anchors: ^ and \$

These don't match characters; they match positions.

- ^ = start of string
- \$ = end of string

Example:

- Pattern: ^cat\$

Matches "cat" only.

- "my cat" → ✗ (extra letters at start)
- "catfish" → ✗ (extra letters at end)

If you don't use ^ and \$, regex is like:

> "Does this appear anywhere inside the string?"

2.5. Quantifiers: +, *, ?, { }

They tell how many times something repeats.

Let's say A is some pattern.

- A+ = 1 or more times
- A* = 0 or more times (can be empty)
- A? = 0 or 1 time (optional)
- A{3} = exactly 3 times
- A{3,6} = between 3 and 6 times (inclusive)
- A{3,} = 3 or more times

Example 1:

- Pattern: \d+
- "1" → ✓

- "1234" → ✓
- "" (empty) → ✗

Example 2:

- Pattern: [A-Za-z]{3,5}
- "ab" → ✗ (too short)
- "abc" → ✓
- "abcd" → ✓
- "abcde" → ✓
- "abcdef" → ✗ (too long)

2.6. Grouping: ()

Parentheses group patterns together.

Example:

(hi)+

Matches:

- "hi"
- "hihi"
- "hihihihi"

2.7. Lookahead: (?=...) (used later for password regex)

Lookahead doesn't consume characters; it just checks that something exists ahead.

- (?=A) means: "From here, the string must contain pattern A starting at this position, but don't move forward."

Example (conceptual):

^(?=.*[A-Z]).+\$

Read as:

- ^ → start
- (?=.*[A-Z]) → somewhere in the string, there must be at least one uppercase letter
- .+ → then match the whole string (any chars)
- \$ → end

So: "String must have at least one uppercase letter, and at least 1 character total."

We'll use this idea for passwords.

3. Deep dive #1: Understanding the email regex

Pattern (JavaScript):

```
let emailReg = /^[\\w.-]+@[\\w.-]+\\. [A-Za-z]{2,6}$ /;
```

We'll break it in chunks:

3.1. ^ - start of string

Says: "Begin matching from the very beginning."

Without ^, something like "xxjohn@gmail.com" would still match, because "john@gmail.com" is inside it.

3.2. [\w.-]+ - the part before the @

[\w.-]+ means:

- [...] → one character that is either:
 - \w → letter / digit / underscore (A-Za-z0-9_)
 - . → a dot
 - - → a hyphen (minus)
- + → one or more of those characters

So the first part:

[\w.-]+

Reads as:

> "One or more characters that are letters, digits, underscore, dot, or hyphen."

Examples that match this part:

- john
- john.doe
- john-doe
- john_doe-99
- a.b-c_d.3

Does not match:

- @john (because first char is @, not allowed)
- john+doe (because + is not in the allowed set here)

3.3. @ - literal at symbol

Just the actual @ character.

So now pattern is:

^[\w.-]+@

Meaning:

> "Start of string, then one or more allowed chars, then an @".

Examples that match so far:

- john@
- john.doe-123@

But we're not done; more parts must follow.

3.4. [\w.-]+ - the domain name part (before the dot)

Same logic as before, but now it's the domain:

```
[\\w.-]+
```

This covers things like:

- gmail
- yahoo
- my-domain
- sub.domain (yes, because we allow dots here too)

So up to now we have:

```
^[\\w.-]+@[\\w.-]+
```

Examples:

- john@gmail → matches up to here.
- john.doe@sub.domain → matches up to here.

Still not "complete email" yet, because we need .com, .org, etc.

3.5. \\ - literal dot

In regex, . (dot) means "any character".

So to match a literal dot, we escape it: \\.

Now the pattern is:

```
^[\\w.-]+@[\\w.-]+\\.
```

Matches things like:

- john@gmail.
- john.doe@sub.domain.

But we still need the last part: .com, .net, .info etc.

3.6. [A-Za-z]{2,6} - the TLD (top-level domain)

```
[A-Za-z]{2,6}
```

Means:

- [A-Za-z] → one letter (upper or lower)
- {2,6} → 2 to 6 letters in a row

So:

- ✓ com (3 letters)
- ✓ net
- ✓ org
- ✓ info (4 letters)
- ✓ museum (6 letters)
- ✗ c (only 1 letter)
- ✗ solutions (9 letters)
- ✗ c0m (digit not allowed)

3.7. \$ - end of string

Now the full pattern:

```
^[\w.-]+@[\w.-]+\.[A-Za-z]{2,6}$
```

Reading it in plain English:

```
> "From the start of the string to the end, it must look like:
> one or more word/dot/hyphen characters, then an @,
> then one or more word/dot/hyphen characters,
> then a dot,
> then 2-6 letters,
> and nothing else before or after."
```

3.8. Test examples

Email	Match?	Why
-----	-----	-----
john@gmail.com	✓	'john' matches first part, 'gmail' matches second, '.com' ok
john.doe99@sub.domain.org	✓	'john.doe99' → [\w.-]+, 'sub.domain' → [\w.-]+, '.org' ok
a@b.co	✓	'a', 'b', '.co'
john@localhost	✗	no dot + TLD at the end
john@@gmail.com	✗	double @ breaks it
@gmail.com	✗	first part empty (we require +)
john@gmail.c	✗	TLD only 1 letter; we need 2-6
john@gmail.comm	✓	'comm' is 4 letters, allowed

So: it's a simplified email regex, but good enough for most basic forms.

4. Deep dive #2: Understanding the password regex

Password pattern (JavaScript):

```
let passReg = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/;
```

We'll go chunk by chunk again.

4.1. ^ - start of string

Same as before: start matching at the beginning.

4.2. (?!.*[A-Za-z]) - must contain at least one letter

This is a lookahead:

- (?! ...) → "From here, the string must satisfy this condition..."
- .* → 0 or more of any chars
- [A-Za-z] → a letter

So:

```
(?=.*[A-Za-z])
```

Means:

> "Looking ahead from the start, somewhere in the string there must be at least one letter."

Examples that pass this condition:

- "abc123" → ✓ (there is at least one letter)
- "123abc" → ✓
- "123" → ✗ (no letters)
- "!!!!" → ✗ (no letters)

But remember: this is only a condition; it doesn't "consume" the characters yet. It just checks.

4.3. (?=.*\d) - must contain at least one digit

Same idea:

- \d = digit
- .* \d = "some characters then a digit somewhere"
- (?!...) = lookahead

So:

(?=.*\d)

Means:

> "Somewhere in the string there must be at least one digit."

Examples that pass this condition:

- "abc123" → ✓
- "password1" → ✓
- "1password" → ✓
- "abcdef" → ✗ (no digit)

4.4. Combining lookaheads

The pattern now is:

^(?=.*[A-Za-z])(?=.*\d)

Meaning:

> "From the start, string must have at least one letter and at least one digit. Order doesn't matter."

Important: These checks don't yet enforce length or allowed characters. They just say: "letters and digit"

4.5. [A-Za-z\d]{8,} - actual characters of the password

Now comes the "real" matching part:

[A-Za-z\d]{8,}

Break it down:

- [A-Za-z\d] = any of:
 - uppercase letter
 - lowercase letter

- digit
- {8,} = at least 8 of those characters

So the actual password must:

- be at least 8 characters long
- contain only letters and digits (no @, #, !, _, etc.)

After the lookaheads, this chunk ensures:

- Length ≥ 8
- Characters $\in \{A-Z, a-z, 0-9\}$

Since we already checked "at least one letter" and "at least one digit", combining all conditions we get:

> "At least 8 characters, only letters and digits, and must contain at least one letter and one digit."

4.6. \$ - end of string

So full password regex:

```
^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$
```

Full English:

> "From start to end, the string must be made of only letters and digits, at least 8 characters long, and

4.7. Test examples

Password	Match?	Why
-----	-----	-----
abc123	✗	length 6 < 8
abcdefgh	✗	no digit
12345678	✗	no letter
abc12345	✓	≥ 8 , has letters & digits
A1b2C3d4	✓	ok
abcd1234!	✗	'!' not allowed (not in [A-Za-z\d])
password1	✓	has letters, 1 digit, len 9

You can now look at any part of a regex and explain what it demands.

5. Using these regex patterns in jQuery validation

Here's a full jQuery example with these patterns and some comments.

File: form.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Register</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
  <form id="regForm" method="post" action="process.php">
```



```

<label>Email:</label><br>
<input type="text" id="email" name="email"><br>
<small id="emailError" class="error"></small><br>

<label>Password:</label><br>
<input type="password" id="password" name="password"><br>
<small id="passError" class="error"></small><br>

<button type="submit">Register</button>
</form>

<script>
$(function () {
    // our regex patterns:
    let emailReg = /^[\\w.-]+@[\\w.-]+\\. [A-Za-z]{2,6}$/;
    let passReg = /^(?=.*[A-Za-z])(?=.*\\d)[A-Za-z\\d]{8,}$/;

    $("#regForm").on("submit", function (e) {
        let email = $("#email").val().trim();
        let password = $("#password").val();

        let valid = true;
        $(".error").text(""); // clear previous errors

        // email check:
        if (!emailReg.test(email)) {
            $("#emailError").text("Please enter a valid email (e.g. name@example.com).");
            valid = false;
        }

        // password check:
        if (!passReg.test(password)) {
            $("#passError").text(
                "Password must be at least 8 characters and contain letters AND numbers."
            );
            valid = false;
        }

        if (!valid) {
            e.preventDefault(); // stop form submission if anything invalid
        }
    });
});
</script>
</body>
</html>

```

Notes:

- .test(value) returns true if regex matches the whole string (because of ^ and \$).
- If any field fails, we show messages and prevent default submission.

6. Doing the same validation in PHP

Everything we did with jQuery should be repeated on the server with PHP.
Never trust only client-side validation.

6.1. PHP functions, arrays, superglobals used

- function clean() → our own helper

- preg_match() → regex in PHP
- filter_var() → built-in email validator (optional extra)
- \$_POST → superglobal for incoming form data
- \$errors = [] → associative array for error messages

6.2. Example process.php

File: process.php

```
<?php
// process.php

// 1. Helper function (PHP function)
function clean($value) {
    return htmlspecialchars(trim($value), ENT_QUOTES, 'UTF-8');
}

// 2. Arrays for data & errors
$errors = [];
$data = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // 3. Superglobal $_POST
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';

    $email = clean($email);

    // 4. Regex patterns (same as JS)
    $emailPattern = '/^[\\w.-]+@[\\w.-]+\\. [A-Za-z]{2,6}$/';
    $passPattern = '/^(?=.*[A-Za-z])(?=.*\\d)[A-Za-z\\d]{8,}$/';

    // Email validation
    if (empty($email)) {
        $errors['email'] = "Email is required.";
    } elseif (!preg_match($emailPattern, $email)) {
        // optionally use filter_var as extra check
        $errors['email'] = "Email format is invalid.";
    } else {
        $data['email'] = $email;
    }

    // Password validation
    if (empty($password)) {
        $errors['password'] = "Password is required.";
    } elseif (!preg_match($passPattern, $password)) {
        $errors['password'] =
            "Password must be at least 8 characters long and contain letters and numbers.";
    } else {
        $data['password'] = $password; // in real app, hash before saving
    }

    // 5. Show result
    if (empty($errors)) {
        echo "<h2>Registration successful</h2>";
        echo "Your email: " . $data['email'];
    } else {
        echo "<h2>Errors:</h2><ul>";
        foreach ($errors as $field => $msg) {
            echo "<li><strong>" . ucfirst($field) . " :</strong> $msg</li>";
        }
    }
}
```

```

        }
        echo "</ul>";
    }

    } else {
        echo "Please submit the form.";
    }
}

```

Notes:

- Functions: `clean()` organizes repeated logic (`trim+escape`).
- Arrays:
 - `$errors['email'] = "..."` stores messages by field name.
 - `$data` stores cleaned/valid values.
- Superglobals:
 - `$_POST` gets user input from form.
 - `$_SERVER['REQUEST_METHOD']` tells if the request is POST.
- Regex in PHP: `preg_match($pattern, $value)`.

7. Extra RegEx practice to make it sink in

Try to write these on your own:

1) Only digits, exactly 4 digits (PIN code)

- Should match: 1234
- Not match: 123, 12345, 12a4

2) Username: starts with a letter, then letters/digits/underscore, 3-15 chars total

- Should match: `user_1`, `a99`
- Not match: `1abc`, `ab` (too short)

3) Phone: may start with +, then digits, length 7-15

- Should match: `+8801712345678`, `01712345678`, `1234567`

4) Password: at least 8 chars, must contain letter, digit and special char `@#$$%`

- Similar to previous password regex, but add one more lookahead for special char.

8. Quick mental checklist for understanding any regex

When you see a scary regex, break it:

1) Look for `^` and `$`

→ Does it try to match the whole string?

2) Split on major structural parts

- Lookaheads (`?=...`)
- Groups (`( ... )`)
- Alternatives `|`

3) Inside each group, read char classes & quantifiers

- `[ ... ]`

- +, *, ?, {m,n}

4) Translate to plain language

- "At least 1 digit"
- "Exactly 3 letters"
- "2 to 6 letters at the end"

Once you do this a few times (like we did for emailReg and passReg), regex stops feeling like magic and b

----- PART 2 - PHP Functions, Arrays & Superglobals

=====

Goals of this part

By the end of this section, you (or your students) should be able to:

- Write and use custom PHP functions (with parameters and return values).
- Use arrays (indexed, associative, multidimensional) to store and work with data.
- Understand and use the main superglobals: \$_GET, \$_POST, \$_SERVER, \$_SESSION, \$_COOKIE, \$_FILES.
- Combine these in a form-processing flow.

1. PHP Functions - your reusable tools

1.1. What is a function?

A function is a named block of code you can call multiple times.

Why we use them:

- Avoid repeating the same code
- Make code cleaner and easier to read
- Separate logic into small pieces

1.2. Basic function syntax

```
function functionName() {
    // code
}
```

Example:

```
function sayHello() {
    echo "Hello, world!<br>";
}

sayHello(); // calls the function
sayHello(); // you can call as many times as you like
```

1.3. Functions with parameters (inputs)

```
function greet($name) {
    echo "Hello, $name!<br>";
}

greet("Emon");
```

```
greet("Sara");
```

- \$name is a parameter (inside the function definition).
- "Emon" and "Sara" are arguments (what you pass in when calling).

Multiple parameters:

```
function add($a, $b) {  
    echo $a + $b . "<br>";  
}  
  
add(5, 3); // 8  
add(10, 7); // 17
```

1.4. Functions that return values

return sends a result back to the caller:

```
function add($a, $b) {  
    return $a + $b;  
}  
  
$result = add(5, 3); // $result = 8  
echo "Sum is: $result";
```

Why return?

Because sometimes you don't want to echo immediately - you want to store or further process the value.

1.5. Default parameter values

If no argument is passed, the default is used:

```
function welcome($name = "Guest") {  
    echo "Welcome, $name!<br>";  
}  
  
welcome(); // Welcome, Guest!  
welcome("Emon"); // Welcome, Emon!
```

1.6. Type hints (optional but good practice)

In modern PHP you can specify types:

```
function multiply(int $a, int $b): int {  
    return $a * $b;  
}  
  
echo multiply(3, 4); // 12
```

- int \$a, int \$b → both parameters must be integers.
- : int after) → function returns an integer.

1.7. A very useful function for forms: clean_input

We use this to sanitize (clean) user input:

```
function clean_input($data) {
```

```

    $data = trim($data);                // remove spaces at edges
    $data = stripslashes($data);        // remove backslashes
    $data = htmlspecialchars($data, ENT_QUOTES, 'UTF-8'); // escape HTML
    return $data;
}

```

Usage:

```
$name = clean_input($_POST['name'] ?? '');
```

This is a perfect example to show how functions make your life easier:
you write once, reuse everywhere.

2. PHP Arrays - storing multiple values in one variable

Arrays are collections.

2.1. Indexed arrays

Indexes are numbers starting at 0.

```

$colors = ["red", "green", "blue"];

echo $colors[0]; // red
echo $colors[1]; // green

// add more
$colors[] = "yellow"; // pushes at the end

```

Looping:

```

foreach ($colors as $color) {
    echo $color . "<br>";
}

```

2.2. Associative arrays

They use keys (like labels) instead of numeric indexes.

```

$user = [
    "name" => "Emon",
    "email" => "emon@example.com",
    "age" => 23
];

echo $user["name"]; // Emon
echo $user["email"]; // emon@example.com

```

Loop with keys and values:

```

foreach ($user as $key => $value) {
    echo "$key: $value<br>";
}

```

Output:

```
name: Emon
```

```
email: emon@example.com
age: 23
```

2.3. Multidimensional arrays

Arrays inside arrays.

```
$students = [
    [
        "name" => "Emon",
        "age"  => 23,
    ],
    [
        "name" => "Sara",
        "age"  => 22,
    ],
];

echo $students[0]["name"]; // Emon
echo $students[1]["age"];  // 22
```

2.4. Arrays for error handling

Very common in form validation:

```
$errors = [];

// some validation
$errors['email'] = "Email is required.";
$errors['password'] = "Password too short.";

if (!empty($errors)) {
    foreach ($errors as $field => $message) {
        echo ucfirst($field) . ": $message<br>";
    }
}
```

Here:

- \$errors is an associative array
- Key → field name (email, password)
- Value → error message

This pattern is super common in PHP apps.

2.5. Useful built-in array functions

You don't need them all now, but good to know:

```
count($arr);           // number of elements
in_array($needle, $haystack); // check if value exists in array
array_keys($arr);       // returns all keys
array_values($arr);     // returns all values
var_dump($arr);         // debug print
print_r($arr);          // nicer debug print
```

Example:

```

$roles = ["admin", "editor", "user"];

if (in_array("admin", $roles)) {
    echo "Has admin role";
}

```

3. PHP Superglobals – special built-in variables

Superglobals are global arrays PHP always provides.
You can access them anywhere (inside or outside functions).

The big ones:

- \$_GET
- \$_POST
- \$_SERVER
- \$_COOKIE
- \$_SESSION
- \$_FILES
- \$_ENV (less common)

Let's look at the main ones you'll use with forms.

3.1. \$_GET – data from URL query string

If you open:

```
page.php?name=Emon&age=23
```

In page.php:

```

$name = $_GET['name']; // Emon
$age  = $_GET['age'];  // 23

```

Great for:

- filters
- pagination (?page=2)
- simple search (?q=php)

3.2. \$_POST – data from submitted forms (method="post")

In HTML:

```

<form action="handle.php" method="post">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>

```

In handle.php:

```

$username = $_POST['username'];
$password = $_POST['password'];

```

\$_POST is used for:

- login

- registration
- profile forms
- anything that changes data

3.3. \$_SERVER - info about the request & server

Very useful values:

```
$_SERVER['REQUEST_METHOD']; // "GET" or "POST"
$_SERVER['PHP_SELF'];       // current script path
$_SERVER['HTTP_USER_AGENT']; // browser info
$_SERVER['REMOTE_ADDR'];    // user IP address
```

Example: only handle form when it's POST:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // process form
} else {
    // maybe show the form or a message
}
```

3.4. \$_COOKIE - small pieces of data stored in the browser

Set a cookie:

```
setcookie("username", "Emon", time() + 3600); // 1 hour
```

Read a cookie:

```
if (isset($_COOKIE['username'])) {
    echo "Welcome back, " . $_COOKIE['username'];
}
```

3.5. \$_SESSION - server-side session storage

First, start session on top of the script:

```
session_start();
```

Set session data:

```
$_SESSION['user_id'] = 5;
$_SESSION['username'] = "Emon";
```

Read it later (on any page after session_start()):

```
echo $_SESSION['username']; // Emon
```

Used heavily for:

- login systems
- shopping carts
- "remember me" states

3.6. \$_FILES - file uploads

HTML:

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="avatar">
  <button type="submit">Upload</button>
</form>
```

PHP:

```
$fileName = $_FILES['avatar']['name'];
$fileTmp  = $_FILES['avatar']['tmp_name'];
$fileSize = $_FILES['avatar']['size'];
```

(That's advanced; just mention it if you want.)

4. Putting it all together: PHP form processing flow

Let's combine functions + arrays + superglobals + regex into a clean flow.

4.1. HTML form (with name, email, password)

File: form.html

```
<form action="process.php" method="post" id="regForm">
  <label>Name:</label><br>
  <input type="text" name="name"><br><br>

  <label>Email:</label><br>
  <input type="text" name="email"><br><br>

  <label>Password:</label><br>
  <input type="password" name="password"><br><br>

  <button type="submit">Register</button>
</form>
```

(You can keep your jQuery validation here if you want—client side.)

4.2. PHP file: process.php

File: process.php

```
<?php
// Start session if you want to store data across pages
session_start();

/**
 * Function to clean input
 */
function clean_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
    return $data;
}

// Arrays for storing cleaned data and errors
```

```

$data    = [];
$errors  = [];

// Only handle when form is submitted via POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // Superglobal $_POST gives us the form fields
    $name    = $_POST['name']    ?? '';
    $email    = $_POST['email']   ?? '';
    $password = $_POST['password'] ?? '';

    // Clean them
    $name    = clean_input($name);
    $email    = clean_input($email);
    // (we don't echo password; just use it for validation)

    // Regex patterns (same as we used in JS)
    $emailPattern = '/^[\\w.-]+@[\\w.-]+\\. [A-Za-z]{2,6}$/';
    $passPattern  = '/^(?=.*[A-Za-z])(?=.*\\d)[A-Za-z\\d]{8,}$/';

    // ---- Validation ----

    // Name: required, at least 2 chars
    if (empty($name)) {
        $errors['name'] = "Name is required.";
    } elseif (strlen($name) < 2) {
        $errors['name'] = "Name must be at least 2 characters.";
    } else {
        $data['name'] = $name;
    }

    // Email: required + regex check
    if (empty($email)) {
        $errors['email'] = "Email is required.";
    } elseif (!preg_match($emailPattern, $email)) {
        $errors['email'] = "Invalid email format.";
    } else {
        $data['email'] = $email;
    }

    // Password: required + regex check
    if (empty($password)) {
        $errors['password'] = "Password is required.";
    } elseif (!preg_match($passPattern, $password)) {
        $errors['password'] = "Password must be at least 8 characters, contain letters and numbers.";
    } else {
        // normally you hash it: password_hash($password, PASSWORD_DEFAULT)
        $data['password'] = $password;
    }

    // ---- After validation ----
    if (empty($errors)) {
        // Example: store name in session
        $_SESSION['username'] = $data['name'];

        echo "<h2>Registration successful!</h2>";
        echo "Welcome, " . $data['name'] . "<br>";
        echo "Your email: " . $data['email'] . "<br>";
        echo "Your IP: " . $_SERVER['REMOTE_ADDR'] . "<br>";

        // Here you could insert into database, redirect, etc.
    } else {
        echo "<h2>There were errors:</h2><ul>";
    }
}

```

```

        foreach ($errors as $field => $message) {
            echo "<li><strong>" . ucfirst($field) . " :</strong> $message</li>";
        }
        echo "</ul>";
        echo '<p><a href="form.html">Go back</a></p>';
    }

} else {
    echo "Please submit the form from form.html";
}

```

What this example shows:

- Functions:
 - clean_input() → reusable cleaning logic.
- Arrays:
 - \$data → valid data.
 - \$errors → associative array of error messages.
- Superglobals:
 - \$_POST → form input.
 - \$_SERVER['REQUEST_METHOD'] → check request method.
 - \$_SERVER['REMOTE_ADDR'] → user IP.
 - \$_SESSION → store username for later.

5. Suggested practice tasks for students

1) Add username field

- Use regex: must start with a letter, allow letters, numbers, underscore, 3-16 chars.
- Validate in jQuery and PHP.

2) Use \$_GET to show a welcome message

- After successful registration, redirect to: welcome.php?name=Emon
- In welcome.php, read \$_GET['name'] and display: "Welcome, Emon!"

3) Store errors in session

- Put \$errors into \$_SESSION['errors'], then redirect back to the form page and show them.

4) Play with arrays

- Create an array of allowed email domains (gmail.com, yahoo.com, etc.).
- Check if user's email domain is in that array using in_array().